

EOSdatabase_demo_v0

June 1, 2026

1 EOS database demo (proof of concept)

1.1 1. Loading stuff

All the code (database class, routine to load data, routines to filter/manipulate data, and plotting routines) is hidden in the `./modules` folder.

```
[1]: from modules.eosdatabase import EOSDatabase
     from modules.functions import *
```

Read all the available thermo data. It will go through all sub-folders to find all files that end with `_thermo.dat`. It will also read the metadata contained in each corresponding `_thermo.toml` file, and it will load that data into the python object. The point is: **developpers add more files, users just need to give the path to the folder.**

Description of EOS stuff is in Section 5.

```
[2]: database = EOSDatabase("./data")
```

```
Loaded: data/solomatovacarcas2021/solomatovacarcas2021_pyrolite_thermo.dat
Loaded: data/miozzi2020/miozzi2020_hcpfe-lowP_thermo.dat
Loaded: data/miozzi2020/miozzi2020_hcpfe-highP_thermo.dat
Loaded: data/caracas2024/caracas2024_pyrolite_thermo.dat
Loaded: data/hakim2018/hakim2018_hcpfe_thermo.dat
```

1.2 2. Printing stuff

I created a little script to easily print the summary of the database that has been loaded. This script is entirely customisable by developpers. Again: **developpers add features, users access the data easily.**

```
[3]: database.summary_thermo()
```

```
EOSDatabase summary thermo
-----
Rows    : 501
Columns: 20
-----
caracas2024 (NaCa2Al3Fe4Mg30Si24O65)
hakim2018 (Fe)
```

```
miozzi2020 (Fe)
solomatovacaracas2021 (NaCa2Al3Fe4Mg30Si24O65)
```

For developpers: all the data is just a giant pandas dataframe. If you are more familiar with pandas dataframes, you can just use them instead.

```
[4]: import pandas as pd
dataframe = database.df_thermo
# print the available keys of the dataframe
print(dataframe.keys())
# print just the P-rho-T-citation info
print(dataframe[["P[GPa]", "rho[g/cm^3]", "T[K]", "entry"]])
```

```
Index(['V[Angstroms^3]', 'rho[g/cm^3]', 'P[GPa]', 'Pst.dev.[GPa]', 'T[K]',
      'Tst.dev.[K]', 'unique_id', 'science_material', 'science_formula',
      'science_phase', 'science_method', 'data_delimiter', 'label', 'entry',
      'drho[g/cm^3]', 'dP[GPa]', 'dT[K]', 'science_cristal', 'MDsteps',
      'U[kJ/kg]'],
      dtype='object')
      P[GPa]  rho[g/cm^3]  T[K]  entry
0      36.200000      4.040000  1994.22  solomatovacaracas2021_pyrolite
1      19.420000      3.530000  1995.31  solomatovacaracas2021_pyrolite
2       8.720000      3.110000  1994.66  solomatovacaracas2021_pyrolite
3       3.110000      2.750000  1995.34  solomatovacaracas2021_pyrolite
4      -0.420000      2.440000  1999.77  solomatovacaracas2021_pyrolite
..      ...          ...      ...      ...
496  121630.849640  105.886608   300.00      hakim2018_hcpfe
497  125204.295132  107.177999   300.00      hakim2018_hcpfe
498  128951.759158  108.501184   300.00      hakim2018_hcpfe
499  132840.026050  109.857449   300.00      hakim2018_hcpfe
500  136910.477163  111.247951   300.00      hakim2018_hcpfe
```

```
[501 rows x 4 columns]
```

1.3 3. Manipulating the database

Here are a few “methods” (python routines) that I vibe-wrote to manipulate the database object. For example, one method prints all the species that exist in the table (material, formula, source).

```
[5]: database.print_species_thermo()
```

```
Species in thermo database
```

```
-----
```

```
iron
```

- Fe : hakim2018
- Fe : miozzi2020

```
pyrolite
```

- NaCa₂Al₃Fe₄Mg₃Si₂₄O₆₅ : caracas2024
- NaCa₂Al₃Fe₄Mg₃Si₂₄O₆₅ : solomatovacarcas2021

Supposedly we will have 500 different entries (500 papers) for 50 different materials, we only want to keep some of it. We have methods to isolate specific information. Here, I coded a script that filters the database in order to keep only things that contain “iron” in the column “material”. In the future, we (developpers) will add more user-friendly filters.

Note: filtering by `field="formula"` and `contains="Fe"` works, but since pyrolite has iron, it will be kept too.

```
[6]: db_iron = database.filter_out(
      field="science_material",
      contains="iron",
    )

db_iron.print_species_thermo()
```

Species in thermo database

iron

- Fe : hakim2018
- Fe : miozzi2020

1.4 4. Outputs: plots and writing to file

Probably one of the most important features of this catalog: be able to write all the data into a file. Again, I went ahead and defined a script making it very easy to do.

```
[7]: db_iron.export_tsv(filename="thermo_iron")
```

Export complete

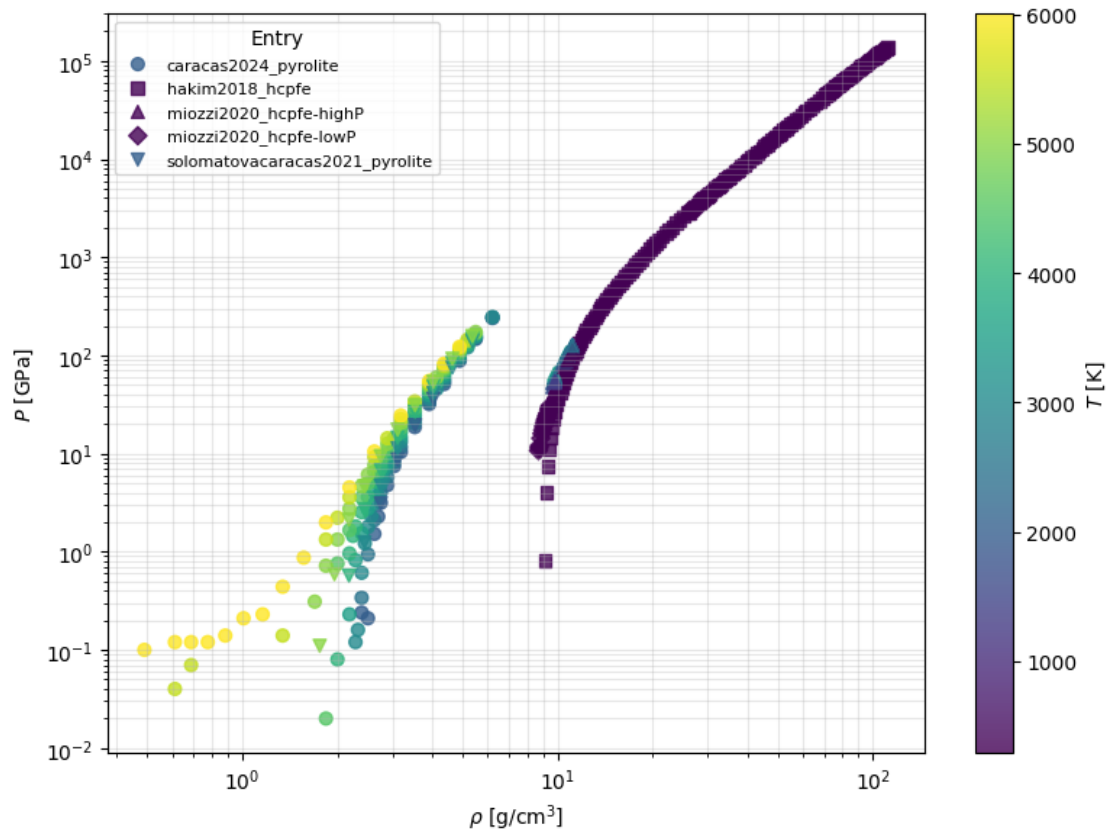
File: output/thermo_iron_20260526.tsv

Rows exported: 310

And I also went a bit crazy and defined a method to do an easy plot: plot all the rho-P-T data, using unique combinations of point styles to distinguish sources.

```
[8]: database.plot_rhopt()
```

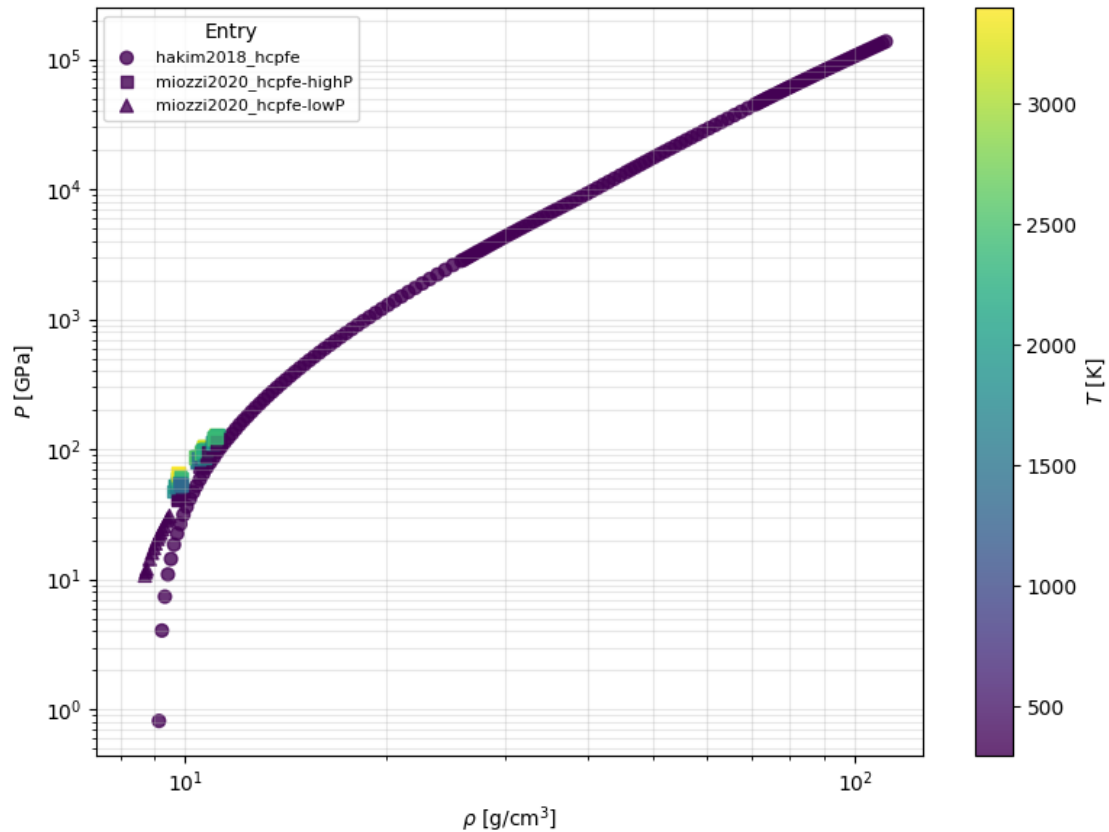
Found 5 entries



Which of course also works for any filtered database, like that of iron.

```
[9]: db_iron.plot_rhopt()
```

Found 3 entries

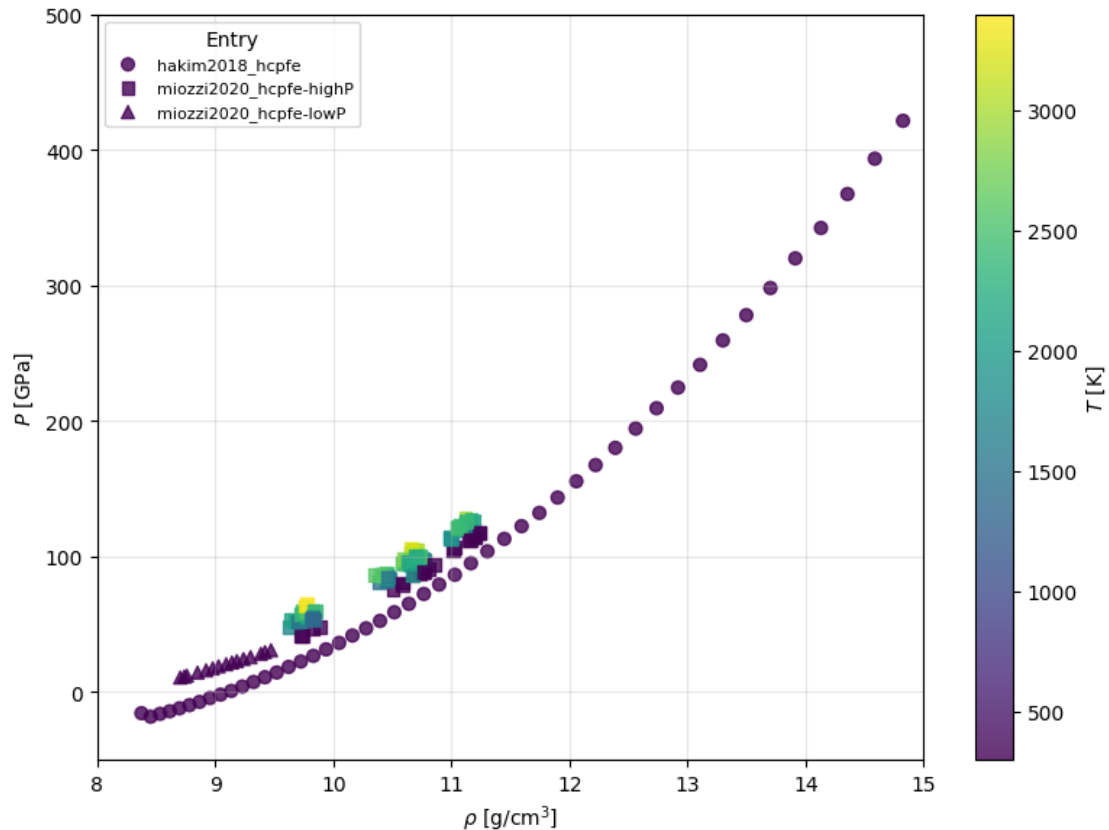


Here I'm just playing around with the plot to make it look nicer. It's really powerful that the entire plot object is just passed to the main program and you can manipulate it however you want...

```
[10]: import matplotlib.pyplot as plt
db_iron.plot_rhoPT(plt_show=False)
plt.xscale('linear')
plt.yscale('linear')
plt.xlim(8,15)
plt.ylim(-50,500)
```

Found 3 entries

```
[10]: (-50.0, 500.0)
```



2 5. Loading EOS and extracting (experimental)

We can apply the same methodology to EOSs. With the same framework, the EOSs were already loaded earlier.

We can define some methods to print summary of available EOS.

```
[11]: database.summary_eos()
```

```
EOSDatabase summary eos
-----
Rows      : 1
Columns: 31
-----
miozzi2020 (Fe)
```

```
[12]: database.print_species_eos()
```

```
Species in eos database
-----
```

```
iron
  - Fe : miozzi2020
```

If we want to work with just one EOS of interest, it's a good idea to extract it from the full catalog. First, search for available entries, then get that entry.

```
[13]: print(database.df_eos["entry"])
```

```
0    miozzi2020_hcpfe
Name: entry, dtype: object
```

```
[14]: miozzi_eos = database.get_eos(entry="miozzi2020_hcpfe")
```

And then use the existing functions to get information from that EOS.

```
[15]: print_science(miozzi_eos)
```

```
Science of entry:  miozzi2020_hcpfe
-----
material: iron
formula: Fe
phase: solid
cristal: hcp
method: DAC
```

```
[16]: print_description(miozzi_eos)
```

```
Description of entry:  miozzi2020_hcpfe
-----
type: fit
cold_type: bm3
thermal_exist: True
thermal_type: debye
reference_temperature: 300
```

```
[17]: print_validity(miozzi_eos)
```

```
Validity of entry:  miozzi2020_hcpfe
-----
min_p: 10
max_p: 150
min_t: 300
max_t: 4000
min_rho: 8
max_rho: 12
```

```
[18]: print_parameters(miozzi_eos)
```

```
Parameters of entry:  miozzi2020_hcpfe
```

```
-----  
V_0: 22.8  
dV_0: 2  
rho_0: 8.7  
drho_0: 0.8  
K_0: 129  
dK_0: 6  
K_0_prime: 6.2  
dK_0_prime: 2  
T_0: 300  
gamma_0: 1.5  
q_debye: 0.8
```

[]: